

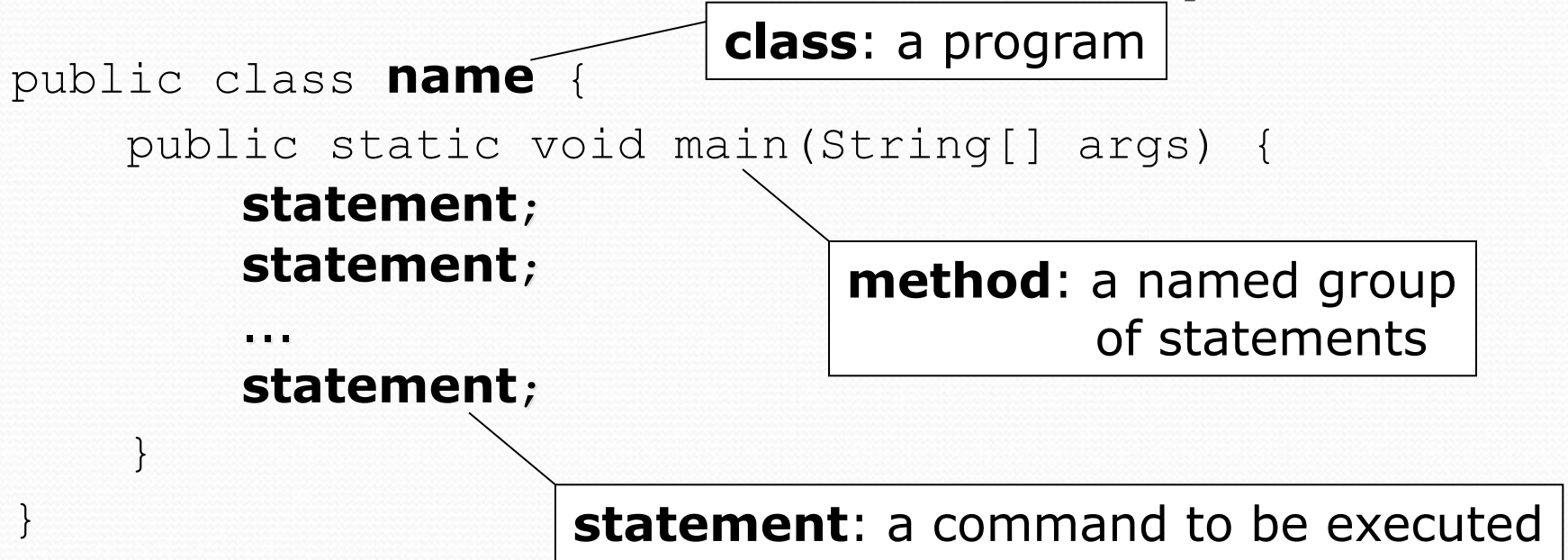


Building Java Programs

Chapter 1

Lecture 1-2: Static Methods

Recall: structure, syntax



- Every executable Java program consists of a **class**,
 - that contains a **method** named `main`,
 - that contains the **statements** (commands) to be executed.

Comments

- **comment:** A note written in source code by the programmer to describe or clarify the code.
 - Comments are not executed when your program runs.
- Syntax:
 - // comment text, on one line**
 - or,
 - /* comment text; may span multiple lines */**
- Examples:
 - // This is a one-line comment.**
 - /* This is a very long
multi-line comment. */**

Where to place comments

- At the top of each file (a "comment header") to describe the program.

```
/* Suzy Student, CS 101, Fall 2019  
   This program prints lyrics about Fraggles Rock. */
```

- At the start of every method (seen later) to describe what the method does.

```
// Print the chorus
```

- To explain complex pieces of code

```
// Compute the Mercator map projection
```

Comments example

```
/* Suzy Student, CS 101, Fall 2019  
   This program prints lyrics about Fraggle Rock. */
```

```
public class FraggleRock {  
    public static void main(String[] args) {  
        // first verse  
        System.out.println("Dance your cares away");  
        System.out.println("Worry's for another day");  
        System.out.println();  
  
        // second verse  
        System.out.println("Let the music play");  
        System.out.println("Down at Fraggle Rock");  
    }  
}
```


Why comments?

- Helpful for understanding larger, more complex programs.
- Helps other programmers understand your code.
 - The “other” programmer could be the future you.

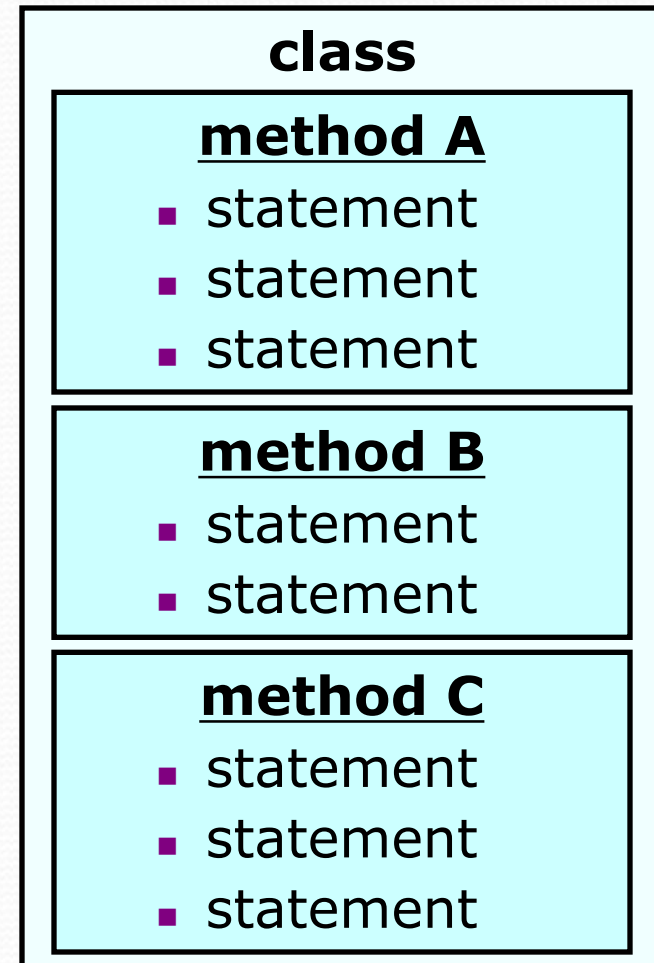


Static methods

reading: 1.4

Static methods

- **static method:** A named group of statements.
 - denotes the *structure* of a program
 - eliminates *redundancy* by code reuse
- **procedural decomposition:**
dividing a problem into methods
- Writing a static method is like
adding a new command to Java.



Using static methods

1. **Design** (think about) the algorithm.
 - Look at the structure, and which commands are repeated.
 - Decide what are the important overall tasks.
2. **Declare** (write down) the methods.
 - Arrange statements into groups and give each group a name.
3. **Call** (run) the methods.
 - The program's `main` method executes the other methods to perform the overall task.

Design of an algorithm

```
// This program displays a delicious recipe for baking cookies.
public class BakeCookies2 {
    public static void main(String[] args) {
        // Step 1: Make the cake batter.
        System.out.println("Mix the dry ingredients.");
        System.out.println("Cream the butter and sugar.");
        System.out.println("Beat in the eggs.");
        System.out.println("Stir in the dry ingredients.");

        // Step 2a: Bake cookies (first batch).
        System.out.println("Set the oven temperature.");
        System.out.println("Set the timer for 10 minutes.");
        System.out.println("Place a batch of cookies into the oven.");
        System.out.println("Allow the cookies to bake.");

        // Step 2b: Bake cookies (second batch).
        System.out.println("Set the oven temperature.");
        System.out.println("Set the timer for 10 minutes.");
        System.out.println("Place a batch of cookies into the oven.");
        System.out.println("Allow the cookies to bake.");

        // Step 3: Decorate the cookies.
        System.out.println("Mix ingredients for frosting.");
        System.out.println("Spread frosting and sprinkles.");
    }
}
```


Declaring a method

Gives your method a name so it can be executed

- Syntax:

```
public static void name() {  
    statement;  
    statement;  
    ...  
    statement;  
}
```

- Example:

```
public static void printWarning() {  
    System.out.println("This product causes cancer");  
    System.out.println("in lab rats and humans.");  
}
```


Calling a method

Executes the method's code

- Syntax:

name () ;

- You can call the same method many times if you like.

- Example:

```
printWarning();
```

- Output:

```
This product causes cancer  
in lab rats and humans.
```

Program with static method

```
public class FreshPrince {  
    public static void main(String[] args) {  
        rap(); // Calling (running) the rap method  
        System.out.println();  
        rap(); // Calling the rap method again  
    }  
  
    // This method prints the lyrics to my favorite song.  
    public static void rap() {  
        System.out.println("Now this is the story all about how");  
        System.out.println("My life got flipped turned upside-down");  
    }  
}
```

Output:

```
Now this is the story all about how  
My life got flipped turned upside-down
```

```
Now this is the story all about how  
My life got flipped turned upside-down
```


Methods calling methods

```
public class MethodsExample {  
    public static void main(String[] args) {  
        message1();  
        message2();  
        System.out.println("Done with main.");  
    }  
  
    public static void message1() {  
        System.out.println("This is message1.");  
    }  
  
    public static void message2() {  
        System.out.println("This is message2.");  
        message1();  
        System.out.println("Done with message2.");  
    }  
}
```

- **Output:**

```
This is message1.  
This is message2.  
This is message1.  
Done with message2.  
Done with main.
```

Control flow

- When a method is called, the program's execution...
 - "jumps" into that method, executing its statements, then
 - "jumps" back to the point where the method was called.

```
public class MethodsExample {  
    public static void main(String[] args) {  
        message1() ;  
  
        message2() ;  
  
        System.out.println("...")  
    }  
    ...  
}
```

```
public static void message1() {  
    System.out.println("This is message1.");  
}
```

```
public static void message2() {  
    System.out.println("This is message2.");  
    message1() ;  
    System.out.println("Done with message2.");  
}
```

```
public static void message1() {  
    System.out.println("This is message1.");  
}
```


When NOT to use methods

- You should not create static methods for:
 - Only blank lines. (Put blank `println`s in `main`.)
 - Unrelated or weakly related statements.
(Consider splitting them into two smaller methods.)

A word about style

- Structure your code properly
- Eliminate redundant code
- Use spaces judiciously and **consistently**
- Indent properly
- Follow the naming conventions
- Use comments to describe code behavior

Why style?

- Programmers build on top of other's code all the time.
 - You shouldn't waste time deciphering what a method does.
- You should spend time on thinking or coding. You should **NOT** be wasting time looking for that missing closing brace.
- So code with style!